

Análises supervisionadas



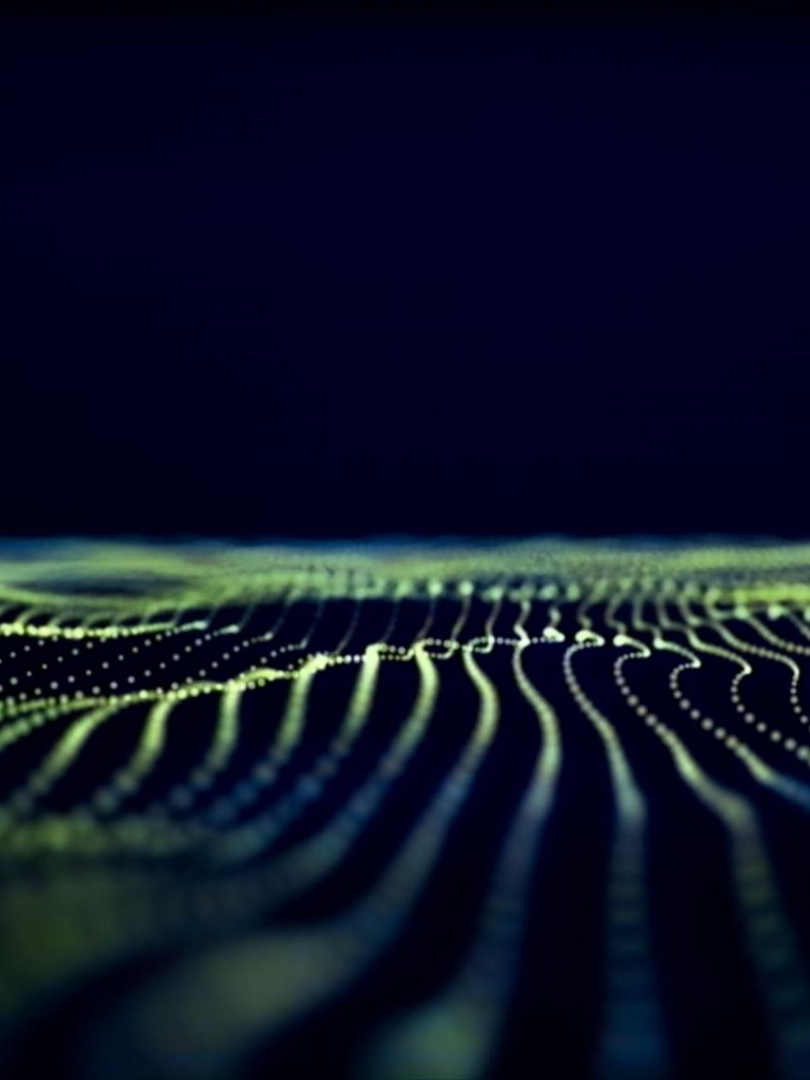


Análises supervisionadas

- São as análises realizadas quando temos um alvo
- Esse alvo, em linguagem de modelagem, é a variável dependente
- O que seria um alvo?
 - Bom/mau pagador
 - Churn (evasão, rotatividade, saída)
 - Aprovado/Reprovado
 - Nota final
 - Salário
 - Lucro anual de uma empresa

Dois tipos principais

- Regressão: alvo numérico, quantitativo (lucro, nota, volume de utilização etc)
- Regressão linear
- Classificação: alvos categóricos ou binários (sim/não, 0/1, favorável/contrário etc)
- Regressão logística
- Árvores de decisão



Regressão linear

- Usado para descrever e prever uma determinada variável alvo - Apenas para dados numéricos

- Fórmula padrão:

$$y = a + \beta x, \text{ onde:}$$

y: é o alvo ou variável dependente

a: é uma constante ou “erro”

β : coeficiente da variável x – parâmetro a ser explicado

x: variável explicativa

- Podemos ter uma longa sequência de variáveis aplicativas, ou seja: $\beta_1 x_1 + \beta_2 x_2 + \beta_n x_n$



Exercitando regressão linear

- Base de Dados:
Data_advertising.csv (fonte: Kaggle)
- Contém 4 variáveis numéricas: TV, Radio, Newspaper representam investimentos nessas mídias e Sales representa a venda daquele período
- Qual é a variável X e a variável Y?

Rodando regressão linear no Excel

- É necessário ter o suplemento de análises estatísticas habilitado
- Apenas com variáveis numéricas
- Limitação da ferramenta: quantidade de dados (linhas)

Rodando regressão linear no Python

- Acessar o Google Colab (link no Moodle). Bibliotecas:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
```

- Comparar com resultados do Excel

Como analisar?

- Primeiro, precisamos identificar a equação resultante:

Y: Sales

a (const): 4,6251

$\beta_1 x_1$: 0,0544 TV

$\beta_2 x_2$: 0,1070 Radio

$\beta_3 x_3$: 0,0003 Newspaper

	coef

const	4.6251
TV	0.0544
Radio	0.1070
Newspaper	0.0003

$$y = 4,6251 + 0,0544 * TV + 0,1070 * Radio + 0,003 * Newspaper$$

Esse modelo é bom?

- Métrica R^2
- Representa a proporção da variância para uma variável dependente (y) que é explicada por uma ou mais variáveis independentes ($x_1, x_2...$)
- Suposição de que **cada** variável independente no modelo contribui para explicar a variação da variável dependente
- Varia de 0 a 1 – Rule of thumb: $> 0,6$
- Rule of thumb: O valor pode ser interpretado como % de explicação!
- Quanto mais próximo de 1 melhor, porém depende...cuidado com Overfitting!



Esse modelo é bom?

- Métrica R^2 ajustado
- Representa a proporção da variância para uma variável dependente (y) que é explicada por uma ou mais variáveis independentes ($x_1, x_2...$), porém...
- Há uma penalização para cada nova variável inserida que não agregue muito ao modelo
- Varia de 0 a 1 – Rule of thumb: $> 0,6$
- Rule of thumb: O valor pode ser interpretado como % de explicação!
- Menor que o R^2 original, melhora o overfitting



Esse modelo é bom?

- Métrica p-value
- O valor p para cada termo testa a hipótese nula de que o coeficiente é igual a zero (sem efeito). Um valor p baixo ($< 0,05$) indica que você pode rejeitar a hipótese nula. Em outras palavras, uma preditora que tenha um valor p baixo provavelmente será uma adição significativa ao seu modelo.
- P baixo? Rule of thumb: $< 0,05$





Tá, mas...esse nosso modelo considerou todas as observações da base

- Estamos falando de aprendizado de máquina, ou seja, implica em treinamento, testagem, verificar o quanto esse modelo acerta as previsões
- Em geral, para fazer isso, dividimos nossa base em base de treinamento e base de teste. Divisão 70/30 ou 60/40 – treinamento/teste
- Como fazer essa divisão?
 - Biblioteca sklearn (scikit.learn):

```
from sklearn.model_selection import train_test_split
```

- Vamos dividir a base em treinamento e teste, indicando as proporções e o método de para essa divisão:

```
X_train, X_test, y_train, y_test =  
train_test_split(X, y, test_size=0.3,  
random_state=42)
```



E como treinar um modelo?

- Depois de dividir, estabelecemos o modelo para a base de treinamento e pedimos para fazer a previsão na base de teste
- Podemos comparar as previsões com os valores real (ver Excel também)
- Análise visual com gráfico de dispersão

Esse modelo é bom? Outras métricas de avaliação

- RMSE – Raiz do Erro quadrático médio: parte das diferenças entre os valores estimados e valores reais da base de teste, penalizando erros maiores. Quanto menor, melhor.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}$$

- MAE – Erro absoluto médio, também parte das diferenças absolutas entre valores estimados e valores reais da base de teste. Quanto menor, melhor. É como se fossem as unidades de distância entre o valor real e o previsto.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

Biblioteca:

```
from sklearn.metrics import mean_absolute_error,  
mean_squared_error
```

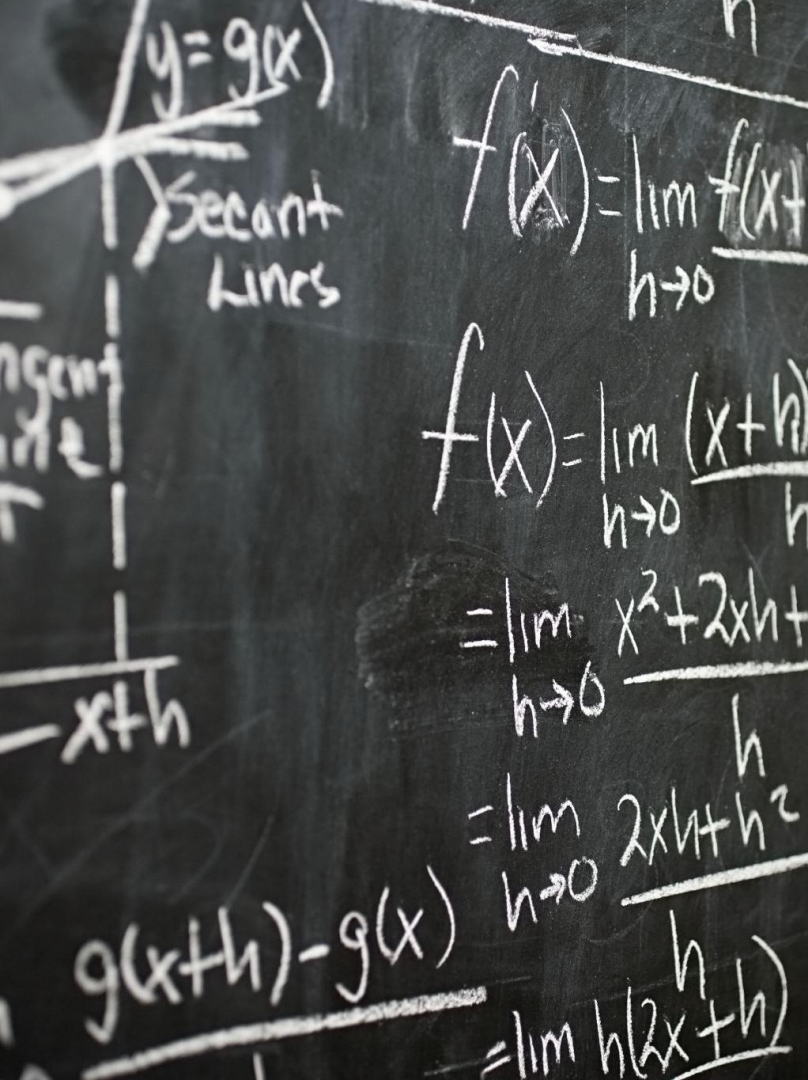


Como prever a nota de um aluno?

- Digamos que a nota depende do esforço, então, a equação seria

$$Nota = a + \beta * esforço$$

- Como determinar esforço?



Base Student Data

Contém dados sobre disciplina de matemática numa escola secundária

- school ID
- gender
- age
- Address (rural/urban)
- Mother education
- Study time
- Father education
- Tempo dedicado aos estudos
- Paid
- Internet
- Absences
- Grades (G1, G2 e G3)

$y = g(x)$
 Secant Lines
 $f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$
 $f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$
 $= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$
 $= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$
 $= \lim_{h \rightarrow 0} h(2x + h)$
 $g(x+h) - g(x)$

E se tivermos variáveis categóricas?

- Se for variável independente, precisaremos transformar, criar *dummies*
- Se for a variável dependente, não é problema para regressão linear!

OneHotEncoder

- OneHotEncoder é uma técnica de pré-processamento comum em aprendizado de máquina, especialmente temos variáveis categóricas.
- Cada valor categórico único é transformado em uma nova coluna binária (0 ou 1).
- Exemplo: Cor pode assumir os valores vermelho, azul e verde.
- Dessa forma, cada categoria é representada por um vetor
- Essa técnica aumenta o número de colunas da base

Cor	Cor_vermelho	Cor_azul	Cor_verde
Vermelho	1	0	0
Azul	0	1	0
Verde	0	0	1

OneHotEncoder

```
from sklearn.preprocessing import OneHotEncoder

# Transformar as variáveis categóricas usando
OneHotEncoder

encoder = OneHotEncoder(drop='first',
                        sparse_output=False)

encoded_categorical_features =
    encoder.fit_transform(categorical_variables)

# Criar um DataFrame com as variáveis categóricas
codificadas

categorical_column_names =
    encoder.get_feature_names_out(input_features=categorical_variables.columns)

categorical_df =
    pd.DataFrame(encoded_categorical_features,
                  columns=categorical_column_names)
```

Cor	Cor_vermelho	Cor_azul	Cor_verde
Vermelho	1	0	0
Azul	0	1	0
Verde	0	0	1



Cor	Cor_azul	Cor_verde
Vermelho	0	0
Azul	1	0
Verde	0	1



Exercício

- Rodar o exemplo
- Qual é a equação resultante?
- Avaliar se é um bom modelo
- Incluir/excluir variáveis com base no p-value e avaliar novamente o modelo
- Como esse modelo poderia ser usado em uma escola? Sugira um sistema ou processo acadêmico/administrativo em que esse modelo poderia ser inserido e usado.
- Postar um relatório em PDF no Moodle.